2.15 Defer

`defer` — это ключевое слово, используемое для отложенного выполнения выражения или функции до момента выхода из текущей функции.

Синтаксис

```
package main

func main() {

    defer println("Завершение программы")
    println("Начало программы")
}
```

Несколько defer

```
package main

func main() {

    defer println("Первый")
    defer println("Второй")
    defer println("Третий")
    println("Основной код")
}
```

Применение в реальных задачах

Закрытие файлов

```
file, err :=
os.Open("test.txt") if
err != nil {
    log.Fatal(err)
}
defer file.Close()
```

MERION

Освобождение ресурсов

```
mu := &sync.Mutex{} mu.Lock()
defer mu.Unlock()
```

Логирование событий

```
func logExecution() {
    fmt.Println("Начало выполнения функции")
    defer fmt.Println("Завершение функции")
    fmt.Println("Выполнение операции")
}

logExecution()
```

MERION

Значения вычисляются в момент вызова

```
package main
import "fmt"

func main() {
x := 10
    defer fmt.Println("Deferred:", x)
    x = 20
    fmt.Println("Current:", x)
}
```

```go
package main
import "fmt"

func main() {
  x := 10

  defer func(n int) {
    fmt.Println("Deferred:", n)
  }(x)

    x = 20
    fmt.Println("Current:", x)
}
```

```go
package main
import "fmt"

func main() {

    x := 10

    defer func(n *int) {
    fmt.Println("Deferred:", *n)
    }(&x)

    x = 20
    fmt.Println("Current:", x)
}
```

Использование анонимных функций для отправки метрик

```
func handle() (err error) {
    defer func() {
        if err != nil {
            increaseErrorCountMetric()
        } else {
            increaseSuccessCountMetric()
        }
    }()

    err = processData()
    if err != nil {
        return // Возвращаем err, так как он уже является именованным параметром
    }

    err = sendRequestToExternalSystem()
    if err != nil {
        return // Возвращаем err, так как он уже является именованным параметром
    }

    ...

    return
}
```

MERION